

Compiler design

1. Lexical analysis:

- Regular expression
- Rules for RE
- Conversion of RE to NFA
- Conversion of NFA with ϵ transition to without ϵ transition NFA
- NFA to DFA conversion
- DFA simulation

Lexical analyser doesn't consider '#include' lines, blank lines, spaces and comments.

eg. int x;

for (i=0; i<=10; i++)

setenv x;

Total tokens = 21

eg. printf("The no.s are %d %d", x, y);

Total tokens = 9

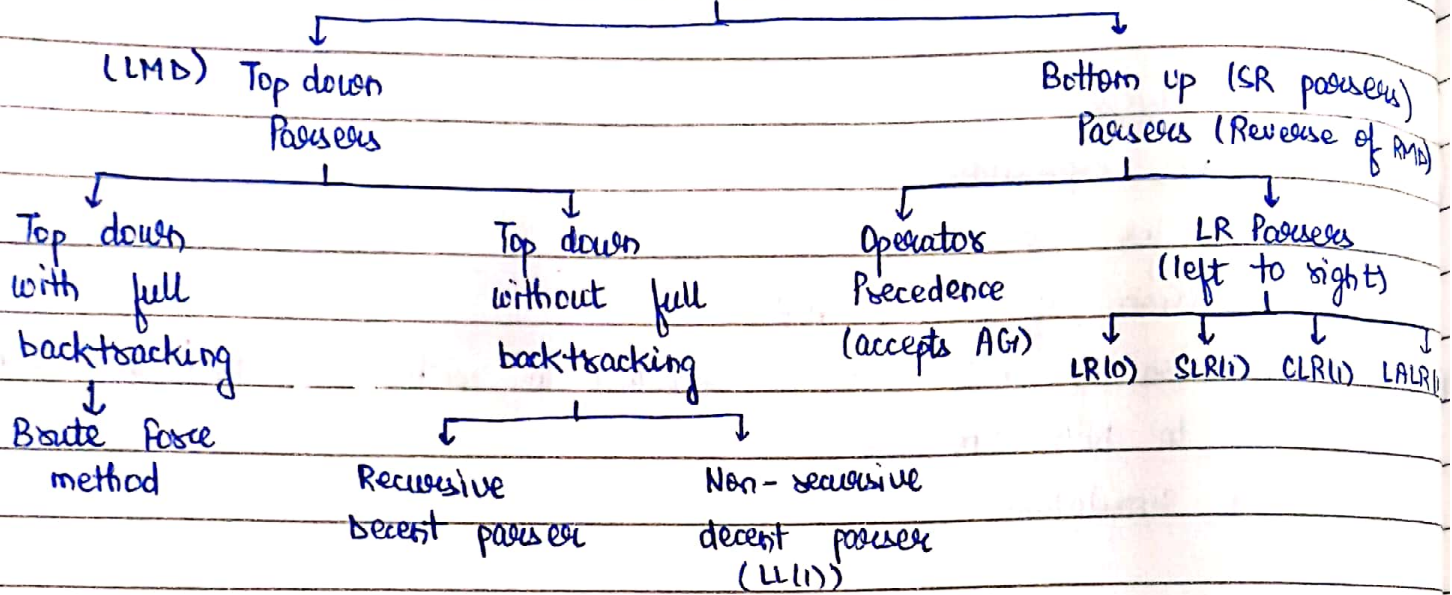
scanf("%d", &x); Total tokens = 8

2. Syntax analysis:

Remove unambiguous elements from grammar.

Remove left recursion.

Parsers



Q. $S \rightarrow aABE$ $w = abbcd$

$A \rightarrow Abc/b$

$B \rightarrow d$

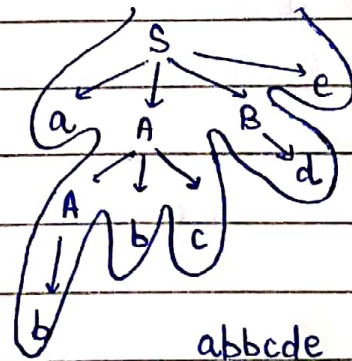
Sol. LMD: Left Most derivation

$S \rightarrow aABe$

$S \rightarrow aAbcBe$

$S \rightarrow abbcBe$

$S \rightarrow abbcd$



RMD: Right Most derivation

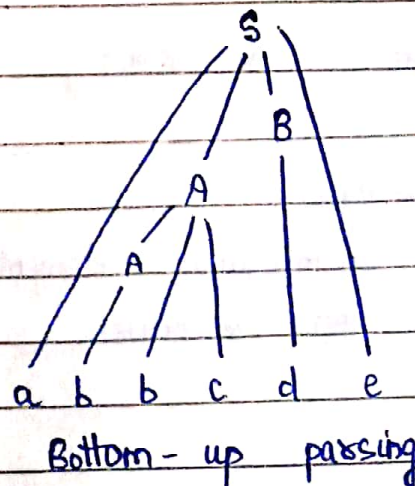
$S \rightarrow aABe$

$S \rightarrow aAde$

$S \rightarrow aAbcde$

$S \rightarrow abbcd$

↑
reverse



Q.
$$\begin{matrix} 2 & 3 & 4 \\ a + b * c \end{matrix}$$

$$E \rightarrow E + E / E * E / id$$

Sol.ⁿ LMD: $E \rightarrow E + E$

$E \rightarrow a + E * E$

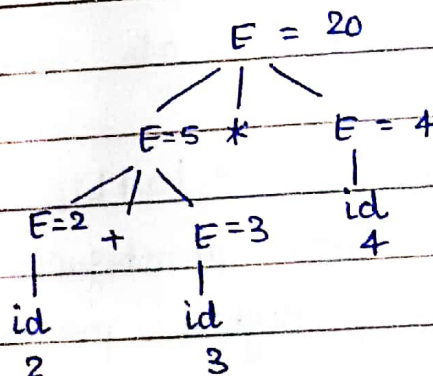
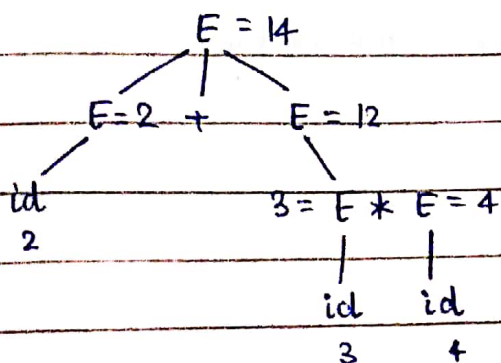
$E \rightarrow a + b * E$

$E \rightarrow a + b * c$

LMD: $E \rightarrow E * E$

$E \rightarrow E + E * E$

$E \rightarrow a + b * c$



Q. $S \rightarrow aSbS / bSaS / E$

$w = abab$

Sol.ⁿ

$S \rightarrow aSbS$

$S \rightarrow a b S a S b S$

$S \rightarrow a b E a S b S$

$S \rightarrow a b a E b S$

$S \rightarrow a b a b E$

$S \rightarrow a b a b$

$S \rightarrow aSbS$

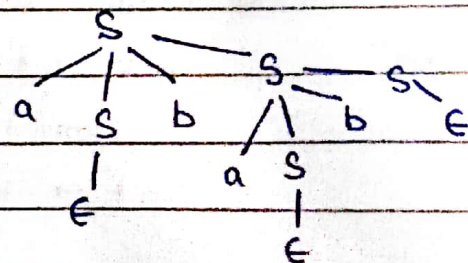
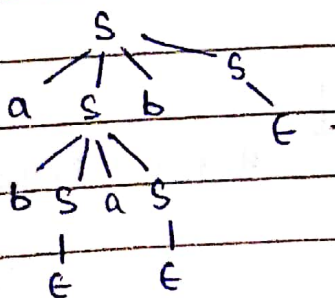
$S \rightarrow a b S$

$S \rightarrow a b a S b S$

$S \rightarrow a b a S b S$

$S \rightarrow a b a b S$

$S \rightarrow a b a b$



Removing ambiguity

$E \rightarrow E + T / T$ left recursive
 $T \rightarrow T * F / F$ left recursive
 $F \rightarrow G \uparrow F / G$ right recursive
 $G \rightarrow id$

Highest precedence gets lowest place in hierarchy.

Q. $R \rightarrow R + R / RR / R^* / a / b / c$

Convert into unambiguous grammar.

Sol.ⁿ R^* Highest precedence
 RR
 $\downarrow$
 $R + R$

~~$R \rightarrow R + R$~~

~~$R \rightarrow R + RR$~~

$E \rightarrow E + T / T$

$T \rightarrow \overbrace{TF}^{\text{to make it left recursive}} / F$

$F \rightarrow F * a / b / c$

Q. $bExp \rightarrow bExp$ and $bExp / bExp$ or $bExp / not\ bExp / True / False$

Convert into unambiguous order.

Sol.ⁿ $E \rightarrow E \text{ or } T / T$
 $T \rightarrow T \text{ and } F / F$
 $F \rightarrow not\ F / True / False$ Highest Precedence
 $\downarrow$

Q. $E \rightarrow E * F$ $* > *$

$E \rightarrow F + E$ $+ < +$

$E \rightarrow F$ $+ = *$

$F \rightarrow F - F / id$ '-' has highest precedence.

Recursion

Left

Right

$$A \rightarrow A\alpha/\beta$$

$$A \rightarrow \alpha A/\beta$$

$$A() \{$$

$$A() \{$$

$$A()$$

$$\alpha$$

$$\alpha$$

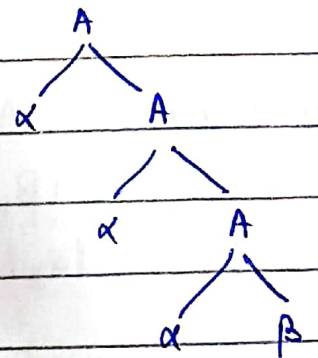
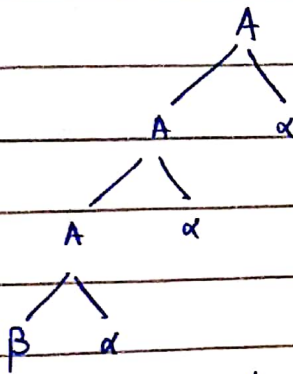
$$A()$$

}

}

It may go into infinite loop.

No infinite loop hence right recursion is not a problem



$\beta\alpha^*$

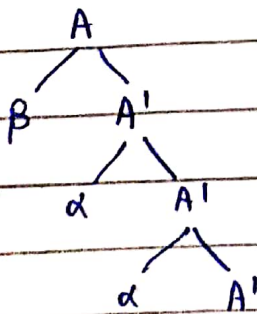
$\alpha^*\beta$

$$A \rightarrow A\alpha/\beta$$

or

$$A \rightarrow \beta A'$$

$$A' \rightarrow \epsilon/\alpha A'$$



right recursive

$$Q. \quad E \rightarrow \underbrace{E}_A + \underbrace{T}_\alpha / \underbrace{T}_\beta$$

Sol.ⁿ

$$E \rightarrow TE'$$

$$E' \rightarrow \epsilon/+TE'$$

Left recursion has been removed.

Q. $S \rightarrow (L) | \alpha$

Sol.ⁿ $\underline{L} \xrightarrow{A} \underline{L} \underline{S} / \underline{S} \xrightarrow{A \quad \alpha \quad \beta} \rightarrow \text{left recursive}$

$L \rightarrow \beta L' \quad \text{or} \quad L \rightarrow SL' \quad (\text{as } \beta = S)$

$L' \rightarrow \epsilon / , SL'$

$S \rightarrow (L) | \alpha$

Q. $\underline{S} \xrightarrow{A} \underline{S} \underline{O} \underline{S} \underline{I} \underline{S} / \underline{O} \underline{I} \xrightarrow{A \quad \alpha \quad \beta}$

Sol.ⁿ $S \rightarrow OIS'$

$S' \rightarrow \epsilon / OSISS'$

Q. $A \rightarrow A\alpha_1 / A\alpha_2 / A\alpha_3 / \beta_1 / \beta_2 / \beta_3$

Sol.ⁿ $A \rightarrow AC / \beta_1 / \beta_2 / \beta_3$

$C \rightarrow \alpha_1 / \alpha_2 / \alpha_3$

$A \rightarrow AC / B$

$B \rightarrow \overset{\alpha}{\beta_1} / \overset{\beta}{\beta_2} / \beta_3$

$C \rightarrow \alpha_1 / \alpha_2 / \alpha_3 \quad \dots ?$

$A \rightarrow BA'$

$A' \rightarrow \epsilon / CA'$

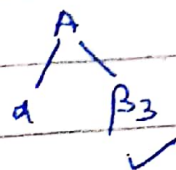
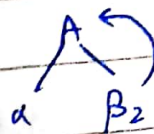
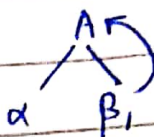
$C \rightarrow \alpha_1 / \alpha_2 / \alpha_3$

$B \rightarrow \beta_1 / \beta_2 / \beta_3$

Non-determinism

$$A \rightarrow \alpha\beta_1 / \alpha\beta_2 / \alpha\beta_3$$

Given: $\alpha\beta_3$



$$A \rightarrow \alpha A'$$

$$A' \rightarrow \beta_1 / \beta_2 / \beta_3$$

(no backtracking required)

Q. $S \rightarrow iETS / iE + SeS / a$

$$E \rightarrow b$$

Sol.ⁿ $S \rightarrow iETS / S' / a$

$$S' \rightarrow \epsilon / eS / a$$

$$E \rightarrow b$$

Q. $S \rightarrow aSSbS / aSaSb / abb / ab$

Sol.ⁿ $S \rightarrow aS' / b$

$$S' \rightarrow SSbS / SaSb / bb$$

Parsers

Q. $E \rightarrow E + T / T$

$$T \rightarrow T * F / F$$

$$F \rightarrow (E) / id$$

Sol.ⁿ $A \rightarrow TA'$

$$A' \rightarrow E / \alpha A'$$

$$\Rightarrow E \rightarrow TE'$$

$$E' \rightarrow \epsilon / +TE'$$

$$T \rightarrow FT'$$

$$T' \rightarrow \epsilon / *FT'$$

$$F \rightarrow (E) / id$$

first()

{ (, id }

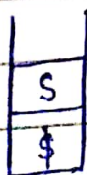
{ +, \epsilon }

{ (, id }

{ *, \epsilon }

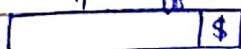
{ (, id }

Parser



Stack

Input Buffer



Parser

Parsing Table

	<u>first()</u>	<u>follow()</u>
Q. $S \rightarrow ABCDE$	$\{a, b, c\}$	$\{\$ \}$
$A \rightarrow a \epsilon$	$\{a, \epsilon\}$	$\{b, c\}$
$B \rightarrow b \epsilon$	$\{b, \epsilon\}$	$\{c\}$
$C \rightarrow c$	$\{c\}$	$\{e, \$, \epsilon, d, e, \$\}$
$D \rightarrow d \epsilon$	$\{d, \epsilon\}$	$\{e, \$\}$
$E \rightarrow e \epsilon$	$\{e, \epsilon\}$	$\{\$ \}$

Soln

	<u>first()</u>	<u>follow()</u>
Q. $S \rightarrow Bb cd$	$\{a, c, b, d\}$	$\{\$ \}$
$B \rightarrow aB \epsilon$	$\{a, \epsilon\}$	$\{b\}$
$c \rightarrow cC \epsilon$	$\{c, \epsilon\}$	$\{d\}$

	<u>first()</u>	<u>follow()</u>
Q. $E \rightarrow TE'$	$\{ (, id \}$	$\{ \$,) \}$
$E' \rightarrow +TE' \epsilon$	$\{ +, \epsilon \}$	$\{ \$,) \}$
$T \rightarrow FT'$	$\{ (, id \}$	$\{ +, \$,) \}$
$T' \rightarrow *FT' \epsilon$	$\{ *, \epsilon \}$	$\{ +, \$,) \}$
$F \rightarrow (E) id$	$\{ (, id \}$	$\{ *, \epsilon, +, \$,) \}$

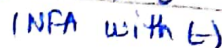
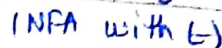
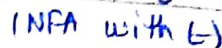
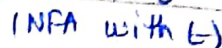
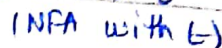
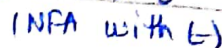
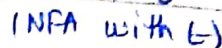
	<u>first()</u>	<u>follow()</u>
Q. $S \rightarrow ACB CbB Ba$	$\{d, h, b, a, g, \epsilon\}$	$\{ \$, a, \epsilon \}$
$A \rightarrow da BC$	$\{d, g, h, \epsilon\}$	$\{h, g, \$\}$
$B \rightarrow g \epsilon$	$\{g, \epsilon\}$	$\{ \$, a, b, g \}$
$c \rightarrow b \epsilon$	$\{h, \epsilon\}$	$\{g, \$, b, h\}$

Q.4. $ab(c|d)^*c$
(NFA with 6)

(NFA with ϵ)

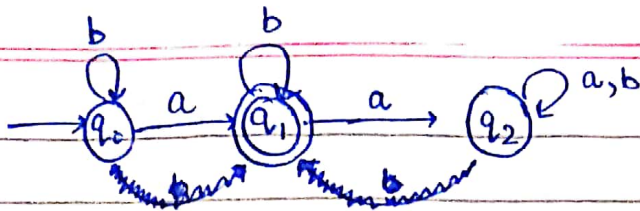
(NFA with ϵ)

(NFA with ϵ)



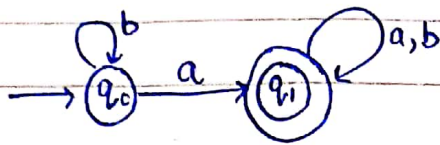
(NFA with ϵ)

Sol. 2.



abbba
bbabb

Sol. 3.



aabbb
bba

Q.5. Remove non-determinism. (common position)

$$S \rightarrow \underline{bSSaaS} / \underline{bSSaSb} / \underline{bSb} / a$$

Sol. n $\alpha \quad \beta_1 \quad \beta_2 \quad \beta_3$

$$A \rightarrow \alpha A'$$

$$A' \rightarrow \beta_1 / \beta_2 / \beta_3$$

$$S \rightarrow bSS' / a$$

$$S' \rightarrow \underline{Saas} / \underline{SaSb} / b$$

$\alpha \quad \beta_1 \quad \beta_2$

$$\Rightarrow S \rightarrow bSS' / a$$

$$S' \rightarrow SaS'' / b$$

$$S'' \rightarrow aS / Sb$$

Recursive decent parsing -

eg. $E \rightarrow E + i i$

Removing left recursion:

$$E \rightarrow iE'$$

$$E' \rightarrow +iE' / \epsilon$$


```

E()
{
    if (l == 'i')
    {
        match('i');
        E();
    }
}

E'()
{
    if (l == '+')
    {
        match('+');
        match('i');
        E'();
    }
    else return;
}

match(char t)
{
    if (l == t)
        l = getch();
    else
        cout << "EOF";
}

```

```

main() {
    E();
    if (l == '$')
        cout << "Parsing successful";
}

```

Q. Design a recursive decent parser for:

Exp $\rightarrow$ digit rest

rest $\rightarrow$ + digit rest / - digit rest / ϵ

digit $\rightarrow$ 0/1/2/-.../9

Trailing (A):

1. If $A \rightarrow \alpha a \beta$ is a production where β is a single non-terminal or ϵ then add 'a' to trailing (A).
2. If $A \rightarrow \alpha B$ is a production then add trailing of (B) to trailing of (A).

	Leading	Trailing
$E \rightarrow E + T$ $\alpha \quad a \quad \beta$	$\{ +, \text{leading}(T) \}$	$\{ +, \text{trailing}(T) \}$
$E \rightarrow T$	$\{ +, *, id, (\}$	$\{ +, *, id,) \}$

$T \rightarrow T * F$ $\alpha \quad a \quad \beta$	$\{ *, \text{leading}(F) \}$	$\{ *, \text{trailing}(F) \}$
$T \rightarrow F$	$\{ *, id, (\}$	$\{ *, id,) \}$

$F \rightarrow id$ $\alpha \quad a \quad \beta$	$\{ id, (\}$	$\{ id,) \}$
$F \rightarrow (E)$ $\alpha \quad a \quad \beta$		
$\alpha \quad a \quad \beta$ $\alpha \quad a \quad \beta$		
$\alpha \quad a \quad \beta$ $\hookrightarrow \text{null}$		

Operator Precedence Parsing Table

1. $\neq \emptyset$ where $\emptyset$: All terminals present in the leading (A) and $\emptyset \neq \emptyset$ for all terminals present in trailing (A).
2. For each production $A \rightarrow X_1 X_2 \dots X_n$ do
for $i=1$ to $n-1$ do
begin
 1. if X_i and X_{i+1} are both terminals then set $X_i = X_{i+1}$
 2. if $i \leq n-2$ and X_i and X_{i+2} are terminals and X_{i+1} is a nonterminal then set $X_i = X_{i+2}$

3. if X_i is a terminal and X_{i+1} is a nonterminal then for all a in $\text{leading}(X_{i+1})$ do set $X_i < a$
4. if X_i is a nonterminal and X_{i+1} is a terminal then for all a in $\text{Trailing}(X_i)$ do set $a > X_{i+1}$

$E \rightarrow E + T$ $\text{leading} \{ +, *, id, (\}$ $\text{trailing} \{ +, *, id,) \}$
 $E \rightarrow T$

$T \rightarrow T * F$ $\{ *, id, (\}$ $\{ *, id,) \}$
 $T \rightarrow F$

$F \rightarrow id$ $\{ id, (\}$ $\{ id,) \}$
 $F \rightarrow (E)$

	+	*	(	)	id	\$
+	>	<	<	>	<	>
*	>	>	<	>	<	>
(	<	<	<	=	<	
)	>	>		>		>
id	>	>		>		>
\$	<	<	<		<	

L-R Parsers (Shift Reduce Parsers)

- Shift (Push)
- Reduce (Pop)
- Accept
- Error

Q. $S \rightarrow aAcBe$ $w = a**A**bbbbcde$

$A \rightarrow Ab/b$

$B \rightarrow d$ Use RMD.

Solⁿ

- $S \rightarrow aAcBe$
- $S \rightarrow a**A**cde$
- $S \rightarrow a**A**bcd$
- $S \rightarrow a**A**bbcd$
- $S \rightarrow a**A**bbbbcde$
- $S \rightarrow abbbbbcde$

A handle of a right sentinel form γ is a production of the form $A \rightarrow B$ and a position of γ where the string B may be found and replaced by A to produce the previous right sentinel form in a rightmost derivation of γ .

Design LL(1) parsing table:

$A \rightarrow \underline{AcB} / \underline{cc/C}$

$B \rightarrow bB / id$

$C \rightarrow \underline{CaB} / \underline{BBB} / \underline{B}$

$A' \rightarrow cCA' / CA'$

$A' \rightarrow \epsilon / cBA'$

$B \rightarrow bB / id$

$C \rightarrow BBBC' / BC'$

$C' \rightarrow \epsilon / aBC'$

first()

$\{c, b, id\}$

$\{\epsilon, c\}$

$\{b, id\}$

$\{b, id\}$

$\{\epsilon, a\}$

follow()

$\{b, a, id, \$, c\}$

$\{b, id, a, \$, c\}$

$\{b, id\}$

$\{b, a\}$

$\{b, id, a\}$

$BC'A'$
 $id a A'$

$BBBC'$
 $id b BC'$

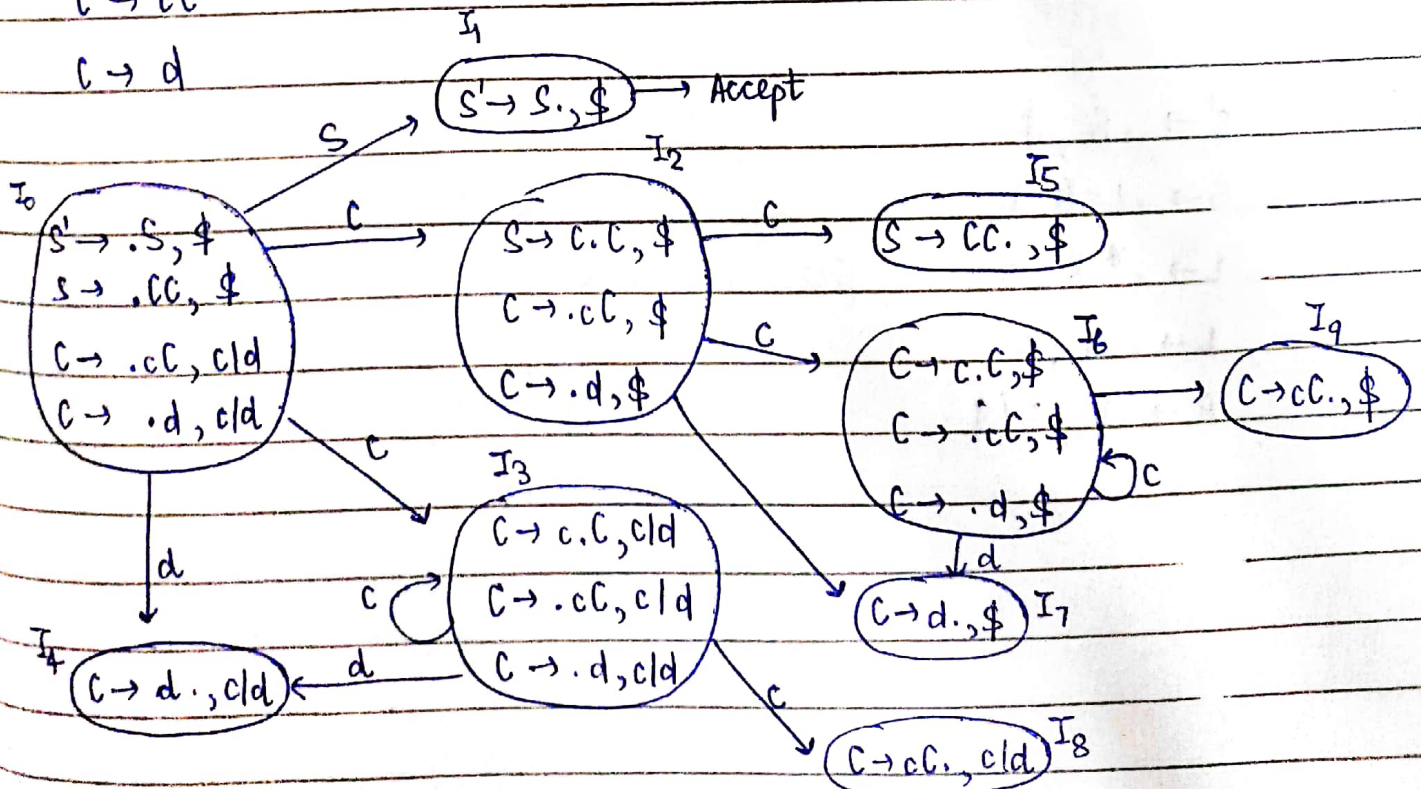
CLR(1) Parsing Table

$S' \rightarrow S$

$S \rightarrow CC$

$C \rightarrow cC$

$C \rightarrow d$



State	c	d	\$	S	C	LALR SLR CLR
0	S ₂	S ₄		1	2	
1			Accept			
2	S ₆	S ₇			5	
3	S ₃	S ₄			8	
4	S ₃	S ₃				
5			S ₁			
6	S ₆	S ₇			9	
7			S ₃			
8	S ₂	S ₂				
9			S ₂			

Q. $S \rightarrow L = R$ Construct CLR(1) parsing table.

$S \rightarrow R$

$L \rightarrow *R$

$L \rightarrow id$

$R \rightarrow L$

Sol.ⁿ

$S' \rightarrow .S, \$$

$S \rightarrow .L = R, \$$

$L \rightarrow .*R, =$

$L \rightarrow .id, =$

$R \rightarrow .L, =/\$$

$S' \rightarrow .S, \$$

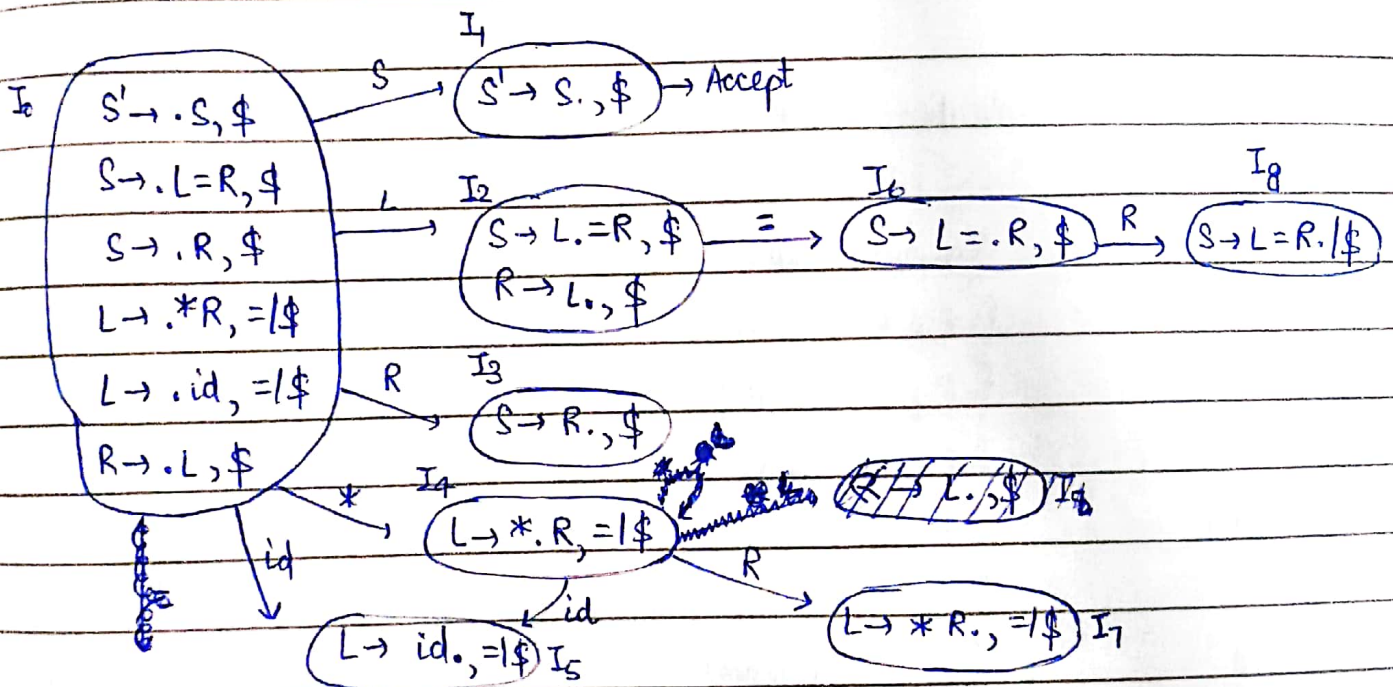
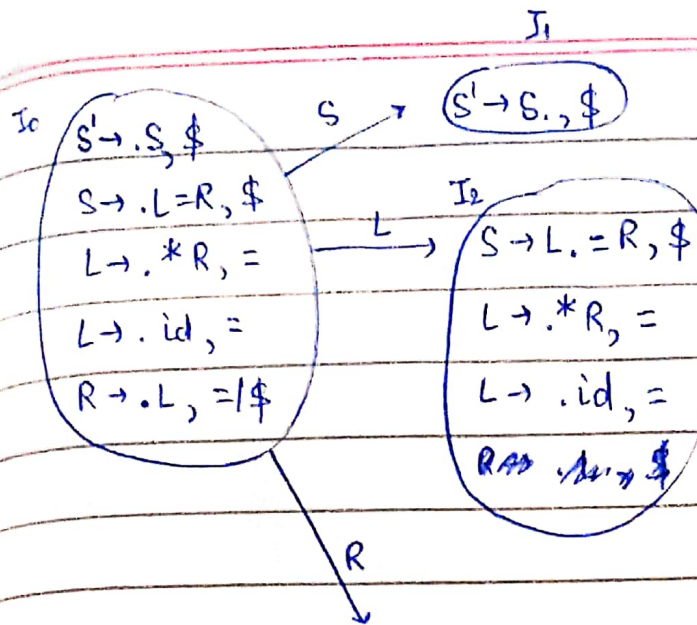
$S \rightarrow .L = R, \$$

$S \rightarrow .R, \$$

$L \rightarrow .*R, =/\$$

$L \rightarrow .id, =/\$$

$R \rightarrow .L, \$$



State = * id \$ S L R

0

1

2

3

4

5

6

7

8

9

Syntax Directed Translation

α + Semantic Rules / $\Rightarrow$ SST
Grammatical Action
Production

SDD: Syntax directed definition

Hides the complexities of the program

No order of evaluation is considered.

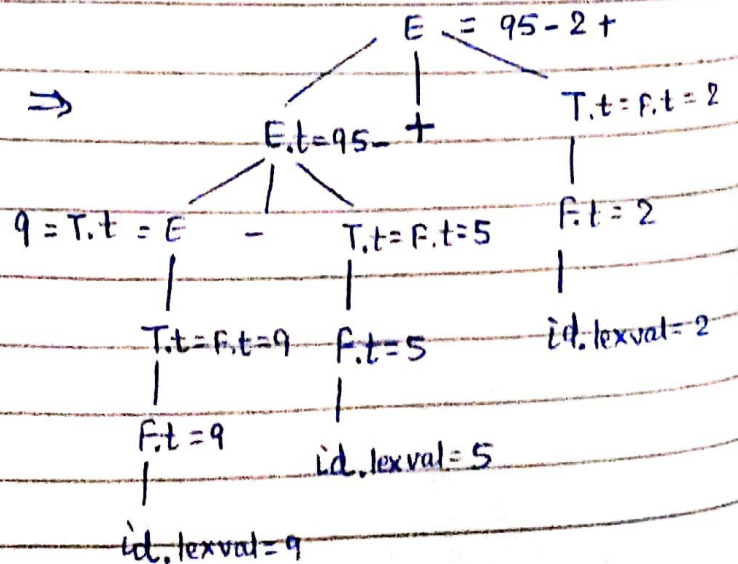
SDT: Order of evaluation matters.

Q. SDD: Semantic Rule:

$$E \rightarrow E + T \quad \{ E_t = E_{1,t} \parallel T_t \parallel ' + ' \}$$
$$E \rightarrow E - T \quad \{ E_t = E_{1,t} \parallel T_t \parallel - \}$$
$$E \rightarrow T \quad \{E, t = T, t\}$$
$$T \rightarrow T * F \quad \{ T_{i,t} = T_{i,t} \parallel F_{i,t} \parallel ' * ' \}$$
$$I \rightarrow F \quad \{I_t = F_t\}$$
$$F \rightarrow \text{id} \quad \{ F_t = \text{id} \cdot \text{lexval} \}$$

Generate parse tree for $9-5+2$.

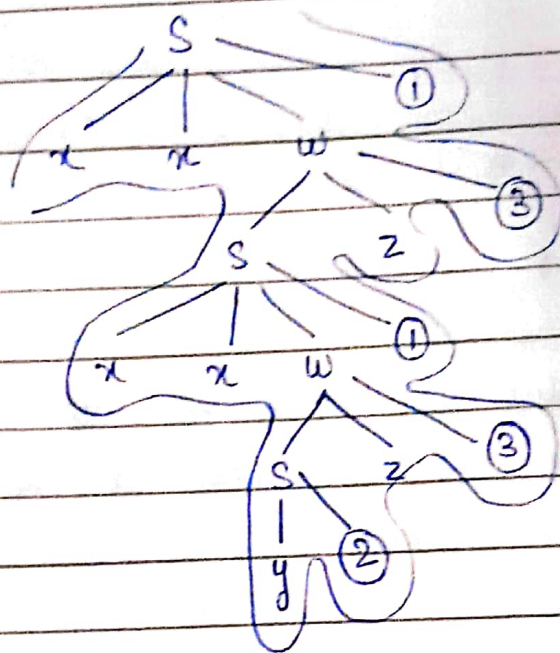
501



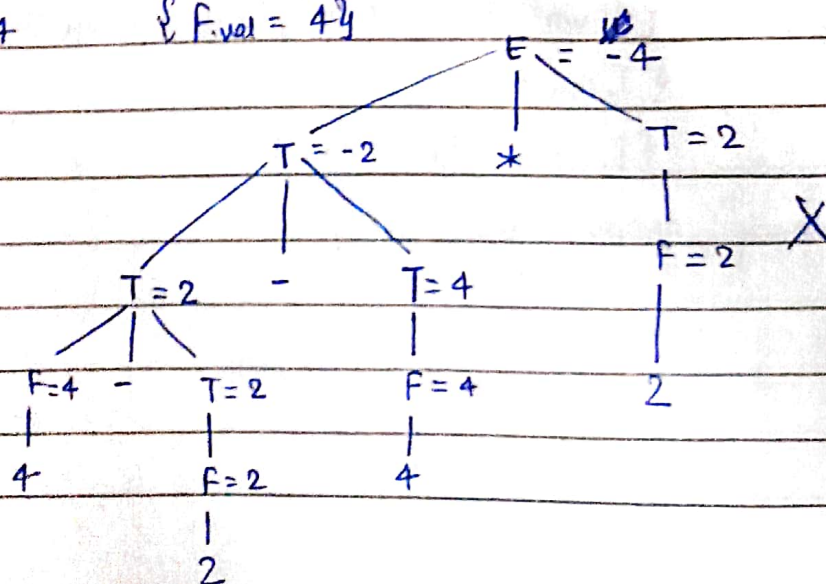
- ① $S \rightarrow xwx$ { Print 6 }
 ③ $w \rightarrow sz$ { Print 2 }
 ② $S \rightarrow y$ { Print 5 }

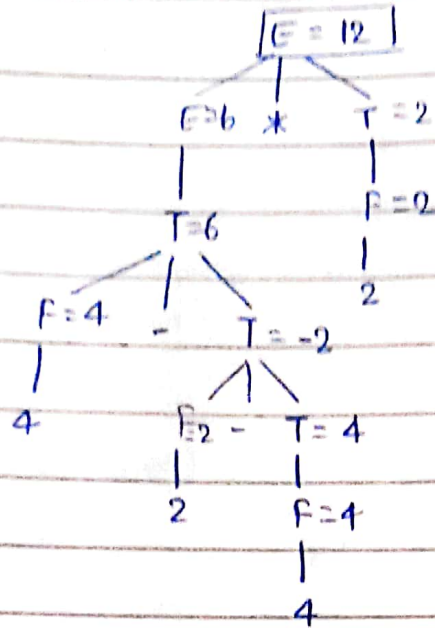
String: xxxxyz

Solⁿ



- ① $E \rightarrow E * T$ { $E.val = E.val * T.val$ } $w = 4 - 2 - 4 * 2$
 ② $/ T$ { $E.val = T.val$ }
 ③ $T \rightarrow F - T$ { $T.val = F.val - T.val$ }
 ④ $/ F$ { $T.val = F.val$ }
 ⑤ $F \rightarrow 2$ { $F.val = 2$ }
 ⑥ 4 { $F.val = 4$ }





Total Reductions = 10

Synthesized Attribute: The value of synthesized attr. at a node is computed from the value of the attr. at the children of that node.

ASSIGNMENT 2:

1. Synthesized & inherited attribute
2. S-attributed & L-attributed grammar } - with eg.
3. How to draw dependency graph from SDT?
4. Given grammar is:

$$T \rightarrow FT' \quad \{ T.inh = F.val ; T.val = T'.syn \}$$

$$T' \rightarrow *FT' \quad \{ T'.inh = T.inh * F.val ; T'.syn = T'.syn \}$$

$$T' \rightarrow \epsilon \quad \{ T'.syn = T.inh \}$$

$$F \rightarrow digit \quad \{ F.val = digit.lexval \}$$

Draw the dependency graph for $3*5$.

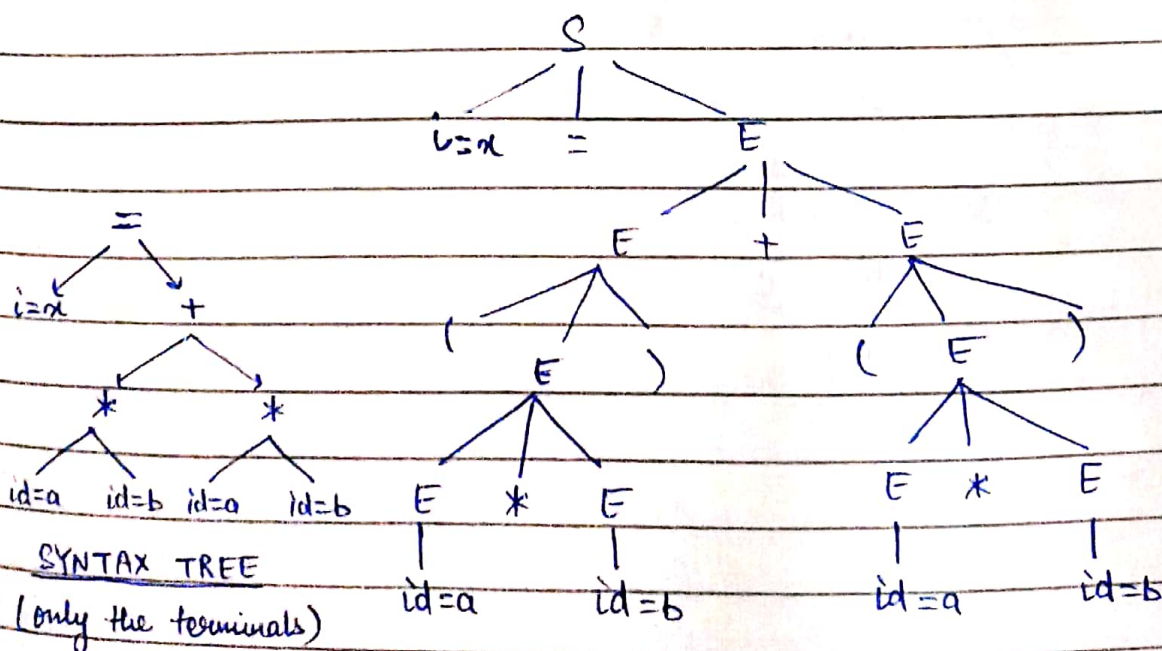
Ans

Representation of Intermediate code

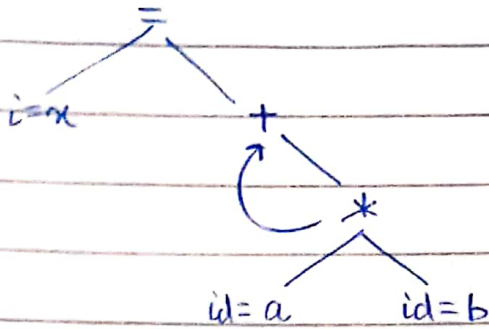
1. Syntax Tree (Abstract Parse Tree)
2. DAG (Directed Acyclic Graph)
3. Postfix
4. Three address code $\rightarrow$ Quadruples
 $\rightarrow$ Triple
 $\rightarrow$ Indirect triple

eg. $x = (a * b) + (a * b)$
 $S \rightarrow i = E$
 $E \rightarrow E + E \mid E * E \mid (E) \mid id \mid E - E$
Construct the parse tree.

Sol.ⁿ $S \rightarrow i = E$
 $\rightarrow i = E + E$
 $\rightarrow i = (E * E) + (E * E)$
 $\rightarrow i = (id * id) + (id * id)$



DAG:



Construct the DAG for the given expression : $a + a * (b - c) + (b - c) * d$

$$x = a + a * (b - c) + (b - c) * d$$

$$S \rightarrow i = E$$

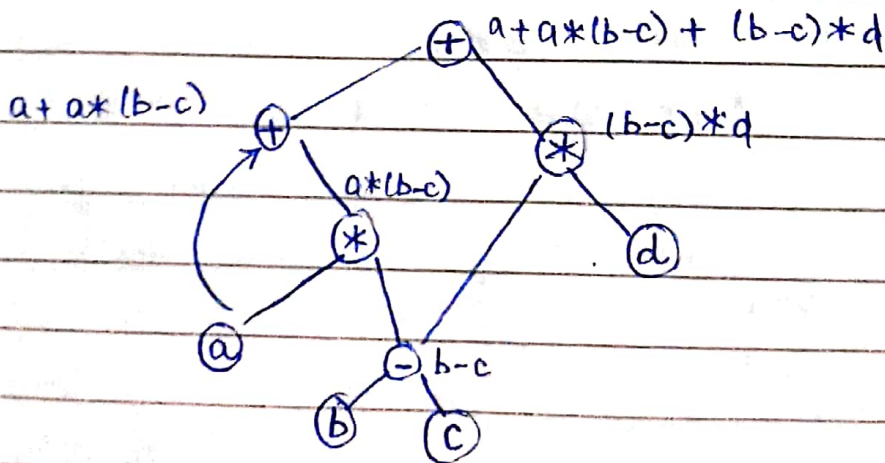
$$\rightarrow i = E + E$$

$$\rightarrow i = id + E + E$$

$$\rightarrow i = id + E * E + E * E$$

$$\rightarrow i = id + id * (E - E) + (E - E) * id$$

$$\rightarrow i = id + id * (id - id) + (id - id) * id$$



DAG

Q. Convert to postfix: (i) If C then A else B.

Sol.ⁿ CAB?

(ii) if C then (if P then Q else R) else B

Sol.ⁿ CPQR? B?

* Quadruple (operator, arg1, arg2, Result):

eg. $A = -B * (C + D)$

$t_1 = -B$

$t_2 = C + D$

$t_3 = t_1 * t_2$

$A = t_3$

	operator	arg1	arg2	result
(0)	unminus	B	-	t_1
(1)	+	C	D	t_2
(2)	*	t_1	t_2	t_3
(3)	=	t_3	-	A

* Triple (using pointers):

	op.	arg1	arg2
(0)	unminus	B	-
(1)	+	C	D
(2)	*	(0)	(1)
(3)	=	(2)	-

Reordering becomes difficult due to pointers.

* Indirect Triple:

Statement

(0) 14

(1) 15

(2) 16

(3) 17

Q. Generate the 3-address code: $x = (b * d) + a - b$

Sol.ⁿ

$$t_0 = b * d$$

$$t_1 = t_0 + a$$

$$t_2 = t_1 - b$$

$$A = t_2$$

(0)	*	b	d	t_0
(1)	+	t_0	a	t_1
(2)	-	t_1	b	t_2
(3)	=	t_2	-	A

(+ and - are left associative.)

Q. Give the 3-address code: If $(A < B)$ then 1 else 0.

Sol.ⁿ

~~Ans in Ans~~

1. If $(A < B)$ then goto (4)
2. $T = 0$
3. goto (5)
4. $T = 1$
5. next

Q. while $(s < d)$
{

if $(x < y)$

$$z = a + b * c$$

else

$$z = 0$$

}

Sol.ⁿ